11. Обектно ориентирано програмиране (на базата на C++ или Java): Основни принципи. Класове и обекти. Конструктори и деструктори. Оператори. Производни класове и наследяване.

(уводни бози)
В днешно време най-разпространените езици за програмиране са т.нар, *езици от високо ниво*. При тях програмистът описва схематично основната идея за решаване на задачата, а специалния програмен транслатор(компилатор или интепретатор) превежда това описание в машинни инструкции за конкретния процесор. Тези езици не зависят от хардуера. Програмирането на високо ниво се свързва с обектно-ориентирания подход – програмиране с помоща на обекти, създването и използването на абстрактен тип данни, наследяването и полиморфизма. Основните характеристики на обектно ориентираното програмиране са:

- Всичко е обект
- Една програма е съвкупснот от обекти, указващи си един на друг чрез изпраяане на съобщения какво да правят
- Обектите притежават собствена памет, суставена от други обекти.
- Всеки обект има тип
- Всички обекти от определен тип могат да получават едни и същи съобщения.

Обектно-ориентираното програмиране се е наложило като стандарт при почти всички съвременни езици за програмиране. То предоставя мощно средство за моделиране на обектите от реалния свят и взаимоотношенията между тях, позволява добро структуриране на програмния код и улеснява неговото преизползване. Благодарение на капсулацията на данните, както и на възможностите за наследяване и работа с абстрактни данни операции, ООП се е утвърдило като предпочитан подход при създаване на големи приложения и библиотеки.

Четирите основни принципа на ООП са капсулация на данните, абстракция, наследяване и полиморфизъм. Те са основните характеристики, които определят един език за програмиране като обектно-ориентиран.

Капсулация на данните
Основна концепция в ООП е обектът да се разглежда като "черна кутия" – използващите обекта "виждат" само атрибутите и операциите, които са присъщи на обекта от реалния свят, без да се интересуват от конкретната им реализация – клиентът на обекта трябва да знае само какво може да прави обектът, а не как го прави. В такъв смисъл "капсулация" означава скриване на ненужните детайли за обектите и откриване към външния свят само на важните техни характеристики и свойства. Обектите в ООП съдържат своите данни и средствата за тяхната обработката, капсулирани като едно цяло.

Абстракция
Абстракцията опростява сложната реалност чрез моделиране на класове подходящи за разрешаването на определен проблем. Тя представлява модел, изглед или някакво друго общо представяне на действителен обект. Абстракцията е разработване на програмен обект представящ обект от реалния свят, а капсулацията му скрива детайлите на имплеметацията. Абстракцията на данните налага ясно разделяне между абстрактните свойства на съответния тип и конкретните детайли на представянето. Идеята е, че промени в това представяне не трябва да имат влияние върху код използващ абстракцията. Нека разгледаме един прост пример: имаме клас Person съответно със

ссвойства Height, Weight, Age и т.н. и действия, които може да се извършват с него: Run(), Sleep(), Cry(). Абстрактното представяне на Person се определя от свойствата които притежава и от действията, които могат да се извършват върху него и от информацията, която може да получим или предадем на него. Конкретната имплементация на действията и свойствата на обекта остава скрита от външния свят, който няма достъп до нея.

Наследяване

Ако един обект съдържа всички свойства и действия на друг, първият може да го наследи. По този начин наследеният обект освен собствените си атрибути и операции приема и тези на "родителя" си (базовия клас), като така се избягва повторното им дефиниране и се позволява създаването на йерархии от класове, моделиращи по естествен начин зависимостите от реалността.

За да изясним това понятие, ще си послужим с класическият в ООП пример за класа от обекти Animal, който представлява абстракция за множеството от всички животни. Всички обекти от този клас имат общи характеристики (например цвят и възраст) и обща функционалност, например операциите Eat и Sleep, докато за класът Dog, представляващ множеството от всички кучета, които също са животни, би могъл да предоставя операциите Eat, Sleep и Bark. Удачно е класът Dog да наследи Animal – тогава той ще съдържа описание само на собственото си действие Bark, докато тези на Eat и Sleep ще получи от базовия си клас. Чрез наследяването се постига специализация, или конкретизация на класовете, тъй като базовият клас представлява категория от обекти по-обща от тази на наследяващите го. Ако си послужим с горния пример, множествата на кучетата и котките са подмножества на множеството от всички животни.

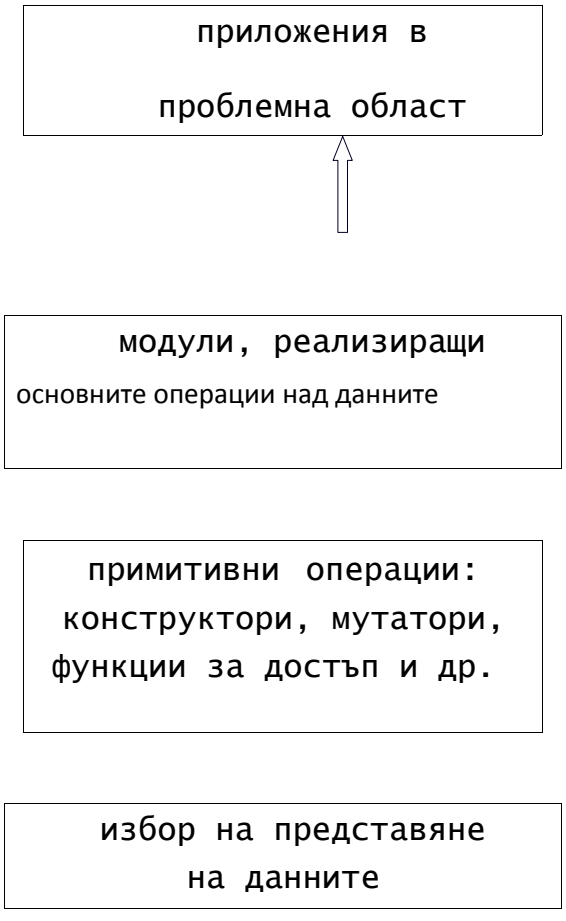
Полиморфизъм

Полиморфизъм буквално означава приемането на различни форми от един обект. Нека е даден базов клас, представящ категория от обекти, които реализират общо действие, което се наследява от множество класове, описващи по-тесни категории. Въпреки, че те всички споделят това действие, те могат да го реализират по различен начин. Когато разполагаме с обекти от базовия клас, знаем че всички те реализират това действие, независимо на кой наследен клас принадлежат. Поради това можем да го използваме без да се интересуваме от конкретната му реализация. Например, ако класът Animal предоставя действието Talk и разгледаме наследяващите го класове Dog и Cat, всеки от тях го реализира по конкрете начин. И ако имаме животно, което издава звук и то е куче – ще лае, а ако е котка – ще мяучи. Полиморфизмът позволява унифицираното извършване на действие над различни обекти, които го реализират. В този случай издаването на звук от животно е полиморфно действие – такова, което се реализира по различен начин в различните наследници на базовия клас.

Класове

Класовете са нови типове данни, дефинирани от потребителя. Те могат да обогатяват възможностите на вече съществуващ тип или да представят напълно нов тип данни. Класовете са подобни на структурите, даже може да се каже, че в някои отношения са почти идентични. В C++ класът може да се разглежда като структура, на която са наложени някои ограничения по отношение на правата на достъп. Отначало ще разгледаме основното, което е приложимо както за класовете, така и за структурите.

Основен принцип на процедурното програмиране е модулния. Програмата се разделя на "подходящи" взаимносвързани части (функции, модули), всяка от които се реализира чрез определени средства. Важен е обаче начинът, по който да става определянето на частите и връзките помежду им. Целта е, следващи промени в представянето на данните да не променят голям брой от модулите на програмата. Разсъждения в тази посока довеждат до подхода абстракция със структури от данни. При него методите за използване на данните се разделят от методите за тяхното конкретно представяне. Програмите се конструират така, че да работят с "абстрактни данни" – данни с неуточнено представяне. След това представянето се конкретизира с помощта на множество функции, наречени конструктори, мутатори и функции за достъп, които реализират "абстрактните данни" по конкретен начин. Така при решаването на даден проблем се оформят следните нива на абстракцията:



Добра реализация на подхода е тази, при която всяко ниво използва единствено средствата на предходното. Предимствата са, че възникнали промени на едно ниво ще се отразят само на следващото над него. Например, промяна на представянето на данните ще доведе до промени единствено на реализацията на някои от конструкторите, мутаторите или функциите за достъп.

Дефиниране на класове

Класовете осигуряват механизми за създаване на напълно нови типове данни, които могат да бъдат интегрирани в езика,

а също за обогатяване възможностите на вече съществуващи типове. Дефинирането на един клас се състои от две части:

- декларация на класа и
- дефиниция на неговите член-функции (методи).

14.2.1. Декларация на клас

Декларацията на клас се състои от заглавие и тяло. Заглавието започва със запазената дума class, следвано от името на класа. Тялото е заградено във фигурни скоби. След скобите стои знакът “;” или списък от обекти. В тялото на класа са декларирани членовете на класа (член-данни и член-функции) със съответните им нива на достъп.

(това изглежда на индиански първоначално, ама като го погледате 10-15 мин се разбира :Д)

Декларация на клас

```
<декларация_на_клас> ::= <заглавие> <тяло>
<заглавие> ::= class [<име_на_клас>]опц
<тяло> ::= {<декларация_на_член>;
            {<декларация_на_член>;}опц
           }[<списък_от_обекти>]опц;
<декларация_на_член> ::=
                              <декларация_на_конструктор>|
<декларация_на_мутатор>|
                         <декларация_на_функция_за_достъп>|
<декларация_на_данна>
<декларация_на_конструктор> ::=
      [<спецификатор_на_достъп>:]опц<име_на_клас>(<парамет
ри>)
<декларация_на_мутатор> ::=
     [<спецификатор_на_достъп>:]опц <тип>
                          <име_на_мутатор>(<параметри>)
<декларация_на_функция_за_достъп> ::=
    [<спецификатор_на_достъп>:]опц <тип>
                   <име_на_функция_за_достъп>(<параметри>)
const;
<спецификатор_на_достъп>  ::=  private  |  public  |
protected
```

```
   <параметри> :: <празно> | void |
                          <параметър> {, <параметър>}опц
   <параметър> ::= <тип> [ &|опц * [const]опц] опц
   <декларация_на_данна>    ::=    <тип>    <име_на_данна>{,
<име_на_данна>}опц
   <тип> ::= <име_на_тип>|<дефиниция_на_тип>
   <списък_от_обекти> ::=
       <обект> [= <име_на_клас>(<фактически_параметри>)]опц
               {,<обект>[=<име_на_клас>(<фактически_парамет
ри>)]опц }опц
              {, <обект>(<фактически_параметри>)}опц
              {, <обект> = <вече_дефиниран_обект>}опц
където  <име_на_клас>,  <име_на_мутатор>,  <име_на_данна>,
<обект> и <име_на_функция_за_достъп> са идентификатори, а
<фактически_ параметри> е определено в Глава 8.
```

Фиг. 14.1 Декларация на клас

За имената на класовете важат същите правила, които се прилагат за имената на всички останали типове и променливи. Също като при структурите името на класа може да бъде пропуснато.

Имената на членовете на класа са локални за него, т.е. в различни класове в рамките на една програма могат да се дефинират членове с еднакви имена. Член-данни от един и същ тип могат да се изредят, разделени със запетая и предшествани от типа им.

Пример:

```
class point
{private:
  double x, y;      // x и y са член-данни
 public:
  point(double, double);
  void read();
  int get_x() const;
  int get_y() const;
  void print() const;
}p=point(2,7), q(-2,3), r=q;
```

В тялото, някои декларации на членове могат да бъдат предшествани от **спецификаторите на достъп** private, public или protected. Областта на един спецификатор на достъп започва от спецификатора и продължава до следващия спецификатор. Подразбиращ се спецификатор за достъп е private. Един и същ спецификатор на достъп може да се използва повече от веднъж в декларация на клас.

Препоръчва се, ако секция public съществува, да бъде първа в декларацията, а секцията private да бъде последна в тялото на класа.

Достъпът до членовете на класовете може да се разгледа на следните две нива:

- По отношение на *член-функциите в класа* е в сила, че те имат достъп до всички членове на класа. При това не е необходимо тези компоненти да се предават като параметри. Този режим на достъп се нарича **режим на пряк достъп**.

- По отношение на *функциите, които са външни за класа*, режимът на достъп са определя от начина на деклариране на членовете.

Членовете на даден клас, декларирани като private (декларирани след запазената дума private) са видими (достъпни) само в рамките на класа. Външните функции нямат достъп до тях. По подразбиране членовете на класовете са private. Чрез използването на членове, обявени като private, се постига скриване на членове за външната за класа среда. Процесът на скриване се нарича още **капсулиране на информацията**.

Членовете на клас, които трябва да бъдат видими извън класа (да бъдат достъпни за функции, които не са методи на дадения клас) трябва да бъдат декларирани като public (декларирани след запазената дума public).

14.2.2 Дефиниране на методите на клас

След декларирането на клас, трябва да се дефинират неговите методи. Дефинициите са аналогични на дефинициите на функции, но името на метода се предшества от името на класа, на който принадлежи метода, следвано от оператора за

принадлежност :: (Нарича се още оператор за област на действие). Такива имена се наричат **пълни.**

Дефиниция на метод на клас
<дефиниция_на_метод_на_клас> ::=
[<тип>]$_{опц}$ <име_на_клас>::<име_на_функция>(<параметри>) [const]$_{опц}$
{<тяло>}
<тяло> ::= <редица_от_оператори_и_дефиниции>
където <име_на_клас> и <име_на_функция> са идентификатори, а <параметри> се определя както в дефиниция на функция.

Дефиниция на метод на клас

Ще отбележим, че дефиницията на конструктор **не започва** с <тип>, а запазената дума const може да присъства само в дефинициите на функциите за достъп. Добрият стил на програмиране изисква използването на const в дефинициите на функциите за достъп и също в техните декларации. Ако се пренебрегне това изискване, могат да се създадат класове, които да не могат да се използват от други програмисти.

Обикновено дефинициите на методите са разположени веднага след декларирането на класа, на който те са членове. Възможно е обаче, дефинициите на методите на един клас да бъдат част от декларациите на този клас, т.е. в декларациите на член-функциите на класа могат да се зададат не само прототипите им, но и техните тела.

Пример: Класът rat може да бъде дефиниран и по следния начин:

```
class rat
{private:
  int numer;
  int denom;
 public:
  rat()
  {numer = 0;
   denom = 1;
  }
  rat(int a, int b)
```

```
 {if (a == 0 || b==0)
{numer = 0;
 denom = 1;
}
  else
  {int g = gcd(abs(a), abs(b));
   if (a>0 && b>0 || a<0 && b<0)
     {numer = abs(a)/g;
      denom = abs(b)/g;}
   else
 {numer = - abs(a)/g;
  denom = abs(b)/g;
 }
  }
 }
void read()
{cout << "numer: ";
 cin >> numer;
 do
 {cout << "denom: ";
  cin >> denom;
 } while (denom == 0);
}
int get_numer() const
{return numer;
}
int get_denom() const
{return denom;
}
void print() const
{cout << numer << "/" << denom << endl;
}
};
```

В този случай обаче член-функциите се третират като **вградени (inline) функции**.

Често член-функциите се реализират като вградени функции. Това увеличава ефективността на програмата, използваща класа.

Ще отбележим също, че в тялото на дефиницията на член-функция явно не се указва обектът, върху който тя ще се приложи. Този обект участва неявно - чрез член-данните на класа. Заради това се нарича **неявен параметър,** а член-данните – **абстрактни данни**.Параметри, които участват явно в дефиницията на член-функция се наричат **явни**. *Всяка член-функция има точно един неявен параметър и нула или повече явни.*

14.2.3 Област на класовете

За разлика от функциите, класовете могат да се декларират на различни нива в програмата: *глобално* (ниво функция) и *локално* (вътре във функция или в тялото на клас).

Областта на глобално деклариран клас започва от декларацията и продължава до края на програмата. Примерите досега бяха с такива класове.

Ако клас е деклариран във функция, всички негови член-функции трябва да са вградени (inline). В противен случай ще се получат функции, дефинирани във функция, което не е възможно.

Пример:

```
void f(int i, int* p)
{int k;
 class CL
 {public:
     // всички методи са дефинирани в тялото на класа
  ...
  private:
  ...
 };
// тяло на функцията f
 CL x;
 ...
}
```

Областта на клас, дефиниран във функция, е функцията. Обектите на такъв клас са видими само в тялото на функцията.

Възможно е използването на обекти (в широкия смисъл на думата) с еднакви имена. В сила е правилото, че в областта си локалният обект скрива нелокалния.

Не е възможно в тялото на локално дефиниран клас да се използва функцията, в която класът е дефиниран.

Пример: Нека сме в означенията на горния пример.

```
void f(...)
{...
 class cl
 {
     // не може да се използва функцията f
 };
 ...
}
```

14.3 Обекти

След като даден клас е дефиниран, могат да бъдат създавани негови екземпляри, които се наричат **обекти**. Връзката между клас и обект в езика C++ е подобна на връзката между тип данни и променлива, но за разлика от обикновените променливи, обектите се състоят от множество **компоненти** (член-данни и член-функции). На Фиг. 14.3 е даден синтаксисът на дефиниция на обект на клас.

Дефиниция на обект на клас

<дефиниция-на_обект_на_клас> ::=
<име_на_клас> <обект>
[=<име_на_клас>(<фактически_параметри>)]$_{опц}$
{,
<обект>[=<име_на_клас>(<фактически_параметри>)]$_{опц}$ }$_{опц}$
{, <обект>(<фактически_параметри>)}$_{опц}$
{, <обект> = <вече_дефиниран_обект>}$_{опц}$;
<обект> ::= <идентификатор>
където <фактически_параметри> е определено в Глава 8.

Фиг. 14.3 Дефиниция на обект на клас

Декларацията на клас не заделя памет за него. Памет се заделя едва при дефинирането на обект от класа. Дефиницията

```
rat p, q(2, 3), r = rat(3, 8);
```

заделя за обектите p, q и r по 8 байта ОП (по 4В за всяка от данните им numer и denom).

Достъпът до компонентите на обектите (ако е възможен) се осъществява чрез задаване на името на обекта и името на данната или метода, разделени с точка. Изключение от това правило правят конструкторите.

Достъп до компонента на обект

<компонента_на_обект> ::= <обект>.<данна>|
<обект>.<име_на_член_функция>()|

<обект>.<име_на_член_функция>(<параметри>)

<име_на_член_функция> е <идентификатор>, означаващ име на мутатор или име на функция за достъп.

Фиг. 14.4 Достъп до компонента на обект

Пример:

```
rat p(1,2), q;
p.get_numer()  // достъп до член-функцията get_numer() за обекта p
q.get_numer()  // достъп до член-функцията get_numer() за обекта q
```

Ще отбележим също, че на практика обектите p и q нямат свои копия на метода get_numer(). И двете обръщения се отнасят за един и същ метод, но при първото обръщение се работи с данните за обекта p, а при второто – с данните за обекта q.

При създаването на обекти на един клас кодът на методите на този клас не се копира във всеки обект, а се намира само на едно място в паметта.

Естествено възниква въпросът *по какъв начин методите на един клас "разбират" за кой обект на този клас са били извикани*. Отговорът на този въпрос дава указателят this. Всяка член-функция на клас поддържа допълнителен формален параметър – указател с име this и от тип <име_на_клас>*.

Обекти от един и същ клас могат да се присвояват един на друг. Присвояването може да е и на ниво инициализация (фиг. 14.3).

Пример: Допустими са дефинициите

```
rat p, q(4,5), r=q;
p = q;
...
r = p;
```

При присвояването се копират всички член-данни на обекта. Така присвояването

```
r = p;
```

е еквивалентно на

```
r.numer = p.numer;
r.denom = p.denom;
```

17.4 Конструктори

Създаването на обекти е свързано с отделяне на памет, запомняне на текущо състояние, задаване на начални стойности и др. дейности, които се наричат **инициализация на обекта.** В езика C++ тези дейности се изпълняват от специален вид член-функции на класовете – конструкторите.

14.4.1. Дефиниране на конструктор

На Фиг. 14.5 е дадена най-често използваната форма за дефиниране на конструктор.

Дефиниране на конструктор (най-често използвана форма)

<дефиниция_на_конструктор> ::=
<име_на_клас>::<име_на_клас>(<параметри>)
{<тяло>}
<тяло> ::= <редица_от_оператори_и_дефиниции>
<параметри> се определя както формални параметри на функция.

Фиг. 14.5 Дефиниране на конструктор (най-често използвана форма)

Ще напомним, че конструкторът е член-функция, която притежава повечето характеристики на другите член-функции, но има и редица особености, като:

- Името на конструктора съвпада с името на класа.
- Типът на резултата е указателят this и явно не се указва.
- Изпълнява се автоматично при създаване на обекти.
- Не може да се извиква явно (обръщение от вида r.rat(1,4) е недопустимо).

Освен това в един клас може явно да не е дефиниран конструктор, но може да са дефинирани и няколко конструктора.

Типична дефиниция на конструктор е дадена в следващия пример.

Пример:

```
class CL
{public:
  CL(int, int, int);
  void print();
  ...
 private:
  int a, b, c;
};
CL::CL(int x, int y, int z) // конструктор с три параметъра
{a = x;
 b = y;
 c = z;
}
```

Класът CL в този пример има един триаргументен конструктор. При създаване на обект на класа, този конструктор ще се изпълнява автоматично, стига да е коректно извикан, в резултат на което член-данните a, b и c на създадения обект ще се свържат със стойностите, които се подадат като фактически параметри на конструктора.

На Фиг. 14.6 е дадена по-обща дефиниция на конструктор.

Дефиниране на конструктор

```
<дефиниция_на_конструктор> ::=
<име_на_клас>::<име_на_клас>(<параметри>):
    <член_данна>(<израз>){,<член_данна>(<израз>)}опц
{<тяло>}
<тяло> ::= <редица_от_оператори_и_дефиниции>
```

Фиг. 14.6 Дефиниране на конструктор

Забелязваме, че е възможно член_данна да се свърже с инициализираща стойност в заглавието на конструктора.

Пример: Конструкторът на класа CL може да се дефинира и по следния начин

```
CL::CL(int x, int y, int z): a(x), b(y), c(z)
{}
```

Ще отбележим, че не е задължително всички член-данни да са инициализирани само пред тялото или само вътре в тялото на конструктора.

Пример: Допустима е дефиницията

```
CL::CL(int x, int y, int z): a(x)
  {b = y;
   c = z;
  }
```

Смисълът на обобщената синтактична конструкция е, че инициализацията на член-данните в заглавието предшества изпълнението на тялото на конструктора. Това я прави изключително полезна. Използването й увеличава ефективността на класа.

14.4.2 Предефинирани конструктори

Обект (в общия смисъл) е предефиниран, ако за него има няколко различни дефиниции, задаващи различни негови интерпретации. За да бъдат използвани такива конструкции е необходим критерии, по който те да се различат.

В рамките на една програма може да се извършва предефиниране на функции. Възможно е:

а) да се използват функции с едно и също име с различни области на видимост

В този случай не възниква проблем с различаването.

б) да се използват функции с едно и също име в една и съща област на видимост

В един клас може да са дефинирани няколко конструктора. Всички те имат едно име (името на класа), но трябва да се различават по броя и/или типа на параметрите си. Наричат се **предефинирани конструктори**. При създаването на обект на класа се изпълнява само един от тях. Определя се съгласно критерия за най-добро съвпадане.

Пример: В класа rat дефинирахме два конструктора rat() и rat(int, int), които се различават по броя на параметрите си.

14.4.3 Подразбиращ се конструктор

В клас може явно да е дефиниран, но може и да не е дефиниран конструктор. Ако явно не е дефиниран конструктор, автоматично се създава един т. нар. **подразбиращ се конструктор**. Този конструктор реализира множество от действия като: заделяне на памет за данните на обект, инициализиране на някои системни променливи и др.

Подразбиращият се конструктор може да бъде предефиниран. За целта е необходимо в класа да бъде дефиниран конструктор без параметри.

Пример: В класа rat, подразбиращият се конструктор беше предефиниран от конструктора

```
rat::rat()
{numer = 0;
 denom = 1;
}
```

14.4.4 Конструктори с подразбиращи се параметри

Функциите в езика C++ могат да имат подразбиращи се параметри. За тези параметри се задават подразбиращи се стойности, които се използват само ако при извикването на функцията не бъде зададена стойност за съответния параметър.

Задаването на подразбираща се стойност се извършва чрез задаване на конкретна стойност в прототипа на функцията или в нейната дефиниция.

При използване на подразбиращи се параметри, важна роля играе редът на параметрите. Прието е, че ако параметър на функция е подразбиращ се, всички параметри след него също са подразбиращи се.

Всичко казано за подразбиращите се параметри на функции се отнася и за конструкторите.

Ако променим дефиницията на класа rat по следния начин:

```
class rat
{private:
  int numer;
  int denom;
 public:
  rat(int=0, int=1);
  void read();
  int get_numer() const;
  int get_denom() const;
  void print() const;
};
```

където конструкторът rat(int, int); се дефинира по същия начин, са допустими следните дефиниции на обекти:

```
rat p,                          // p се инициализира с 0/1
      q = rat(),                // q се инициализира с 0/1
      r = rat(5),           // r се инициализира с 5/1
      s = rat(13,21),       // s се инициализира с 13/21
      t(2);                     // t се инициализира с 2/1
```

14.4.5 Конструктори за присвояване и копиране

Инициализацията на новосъздаден обект на даден клас може да зависи от друг обект на същия клас. За да се укаже такава зависимост се използва знакът за присвояване или кръгли скоби.

Пример: Нека

```
rat p = rat(1,4);
```

Чрез еквивалентните конструкции

```
rat q = p;
rat q(p);
```

се създава обектът q от клас rat, като инициализацията на q зависи от p. Тази инициализация се създава от специален конструктор, наречен **конструктор за присвояване.**

Конструкторът за присвояване е конструктор, поддържащ формален параметър от тип <име_на_клас> const &.

- Ако в един клас явно не е дефиниран конструктор за присвояване, компилаторът автоматично създава такъв, в момента когато новосъздаден обект се инициализира с обекта, намиращ се от дясната страна на знака за присвояване или в кръглите скоби. Този конструктор за присвояване се нарича конструктор за копиране.

Дефиницията rat p=q създава нов обект p *(без викане на конструктор)*, в който се копират съответните стойности на обекта q. Това е резултат от изпълнението на обръщението към rat(&p, q).

- Ако в класа е дефиниран конструктор за присвояване, компилаторът го използва.

Предефиниране на служебния конструктор за копиране се налага когато обектите имат член-данни, указващи динамична памет.

Освен в горните случаи, конструкторът за присвояване (копиране) се използва и при предаване на обект като аргумент на функция, когато предаването е по стойност, а също при връщане на обект като резултат от изпълнение на функция.

14.5 Указатели към обекти на класове

Дефинират се по същия начин както се дефинират указатели към променливи от стандартните типове данни.

Пример:

```
rat p;
rat * ptr = &p;
```

В резултат ще се отделят 4В ОП, които ще се именуват с ptr и ще се инициализират с адреса на обекта p.

Достъпът до компонентите на рационалното число, сочено от ptr, се осъществява по общоприетия начин:

```
(*ptr).get_numer()

(*ptr).get_denom()
```

Синтактичната конструкция (*ptr). е еквивалентна на ptr ->. Така горните обръщения могат да се запишат и по следния начин:

```
ptr -> get_numer()

ptr -> get_denom()
```

14.6 Масиви и обекти

Елементите на масив могат да са обекти, но разбира се от един и същ клас. Дефинират се по общоприетия начин (Фиг. 14.7).

Дефиниция на масив от обекти

<дефиниция_на_променлива_от_тип_масив_от_обекти> ::=

T <променлива>[size] [= {<инициализиращ_списък>}]$_{опц}$;

където

- T е име или декларация на клас;
- <променлива> е идентификатор;
- size е константен израз от интегрален или изброен тип с *положителна* стойност;
- <инициализиращ_списък> се дефинира по следния начин:

<инициализиращ_списък> ::= <стойност>{, <стойност>}$_{опц}$

```
                {,
<име_на_конструктор>(<фактически_параметри>)}опц
```

Фиг. 14.7 Дефиниция на масив от обекти

Пример:

```
rat table[10];
```

определя масив от 10 обекта от клас rat.

Достъпът до елементите на масива е пряк и се осъществява по стандартния начин – чрез индексирани променливи.

Пример: Чрез индексираните променливи

```
table[0], table[1], ..., table[9]
```

се осъществява достъп до първия, втория и т.н. до десетия елемент на table.

Тъй като table[i] (i = 0, 1, ..., 9) са обекти, възможни са следните обръщения към техни компоненти:

```
table[i].read();              // въвежда стойност на table[i]
table[i].print();             // извежда стойността на table[i]
table[i].get_numer(); // намира числителя на table[i]
table[i].get_denom();    // намира знаменателя на table[i].
```

Връзката между масиви и указатели е в сила и в случая когато елементите на масива са обекти. Името на масива е указател към първия му елемент, т.е. ако

```
rat * p = table;         // p сочи към table[0]
                                    // т.е. p==&table[0]
```

*(p+i) == table[i], i = 0, 1,...,9

Тогава

(*(p+i)).print(); // е еквивалентно на table[i].print();

Масивът може да е член-данна на клас. Конструкторите (в частност конструкторът по подразбиране) играят важна роля при дефинирането и инициализирането на масиви от обекти. Масив от обекти, дефиниран в програма, се инициализира по два начина:

- *неявно* (чрез извикване на системния конструктор по подразбиране за всеки обект – елемент на масива);
- *явно* (чрез инициализиращ списък).

Примери:

а) Класът

```
const NUM = 5;
class student
{public:
 void read_student();
 void print_student() const;
 bool is_better(student const &) const;
 double average() const;
 private:
 int facnom;
 char name[26];
 double marks[NUM];
};
```

няма явно дефиниран конструктор. Дефиницията

```
student table[30];
```

на масива table от 30 обекта от клас student е правилна.

Инициализацията се осъществява чрез извикване на “системния” конструктор по подразбиране за всеки обект – елемент на масива.

б) Класът rat, дефиниран по-долу

```
class rat
{private:
  int numer;
  int denom;
 public:
  rat(int=0, int=1);
  void read();
  int get_numer() const;
  int get_denom() const;
  void print() const;
};
```

притежава явно дефиниран конструктор с два подразбиращи се параметъра. В този случай са допустими дефиниции от вида:

```
rat x[10]; // x[i] се инициализира с 0/1, за всяко i=0,1,...9.
rat x[10] = {1,2,3,4,5,6,7,8,9,10}; //x[i] == i/1
rat x[10] = {rat(1,21),rat(2),rat(3, 5),4,5,6,7,8,9,10};
// x[0] == 1/21; x[1] == 2/1; x[2] == 3/5, x[3]== 4/1, ...
```

Ако променим конструктора на класа rat от

```
rat(int=0, int=1);
```

в

```
rat(int, int);
```

т.е. без подразбиращи се параметри и трите дефиниции от по-горе ще съобщят за грешка. Единствено допустима дефиниция на x[10] е с инициализация с 10 обръщения към двуаргументния конструктор rat с явно указани два аргумента.

Динамични обекти

Вече разгледахме в най-общ план разпределението на ОП по време на изпълнението на програма. От Фиг. 8.1 на Глава 8 се вижда, че всяка програма има две "места" за памет: *програмен стек (стек)* и *област за динамичните данни (динамична памет, хийп или куп).*

Стекът е област за временно съхранение на информация. Той е кратковременна памет. C++ използва стека основно за реализиране на обръщения към функции. Всяко обръщение към функция предизвиква конструиране на нова стекова рамка, която се установява на върха на стека. По такъв начин когато функция A извика функция B, която от своя страна вика функция C, стекът нараства. Когато пък всяка от тези функции завършва, стековите рамки на тези функции автоматично се разрушава. Така стекът се свива.

Хийпът е по-постоянна област за съхранение на данни. Той е един вид дълготрайна памет. Особеност на тази памет е, че тя не се свързва с имена на променливи. С разположените в нея обекти се работи косвено – чрез указатели. Обикновено се използва при работа с т. нар. **динамични структури от данни**. *Динамичните данни са такива обекти (в широкия смисъл на думата), чийто брой не е известен в момента на проектирането на програмата. Те се създават и разрушават по време на изпълнението на програмата. След разрушаването им, заетата от тях памет се освобождава и може да се използва отново. Така паметта се използва по-ефективно.*

Създаването и разрушаването на динамични обекти в C++ се осъществява чрез операторите new и delete. Извикването на new заделя в хийпа необходимата памет и връща указател към нея. Този указател може да се съхрани в някаква променлива и да се пази докато е необходимо. За разлика от стека,

заделянето на памет в хийпа е явно – чрез new. Освобождаването на паметта от хийпа също става явно, чрез delete. Всяко извикване на new трябва да бъде балансирано чрез извикване на delete. Последното се налага, тъй като за разлика от стека, хийпът не се изчиства автоматично. В C++ няма система за "събиране на боклуци" (автоматично премахване на обекти, които вече не са необходими). Затова трябва явно да бъдат изтрити създадените в хийпа обекти. Описанието на оператора new е дадено на Фиг. 14.8.

Оператор new

Синтаксис

new <име_на_тип> [[size]]$_{опц}$;|

new <име_на_тип> (<инициализация>);

където

- *<име_на_тип>* е име на някой от стандартните типове int, double, char и др. или е име на клас;

- *size* е израз с произволна сложност, но трябва да може да се преобразува до цял. Показва броя на компонентите от тип <име_на_тип>, за които да се задели памет в хийпа и се нарича **размерност**;

- *<инициализация>* е израз от тип <име_на_тип> или инициализация на обект според синтаксиса на конструктора на класа, ако <име_на_тип> е име на клас.

Семантика

Заделя в хийпа (ако е възможно):

- sizeof(<име_на_тип>) байта, ако не са зададени size и <инициализация> или

- sizeof(<име_на_тип>)*size байта, ако явно е указан size или

- sizeof(<име_на_тип>) байта, ако е специфицирана <инициализация>, която памет се инициализира с <инициализация>

и връща указател към заделената памет.

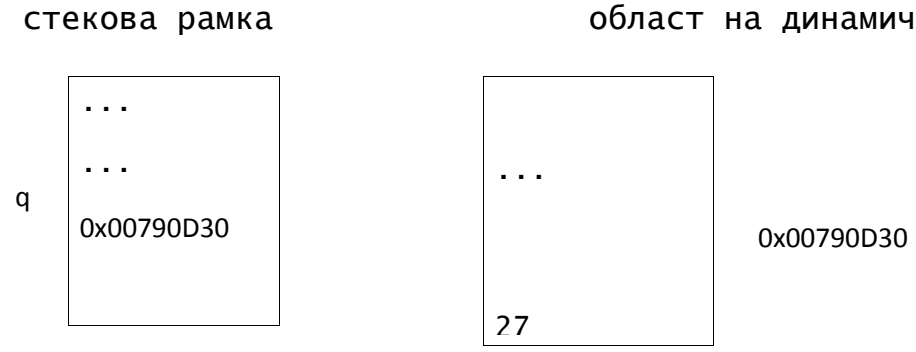
Ако няма достатъчно памет в хийпа, операторът new връща NULL.

Фиг. 14.8 Оператор new

Примери:

a) int* q = new int(2+5*5);

отделя (ако е възможно) 4B памет в хийпа, инициализира я с 27 - стойността на израза 2+5*5 и свързва q с адреса на тази памет, т.е.



Заделянето на памет по време на компилация се нарича **статично** заделяне на памет, заделянето на памет по време на изпълнение на програмата - **динамично разпределение на паметта**.

Под период на **активност на една променлива** се разбира частта от времето за изпълнение на програмата, през което променливата е свързана с конкретно място в паметта. Паметта за глобалните променливи се заделя в началото и остава свързана с тях до завършването на изпълнението на програмата. Паметта за локалните променливи се заделя при влизане в локалната област и се освобождава при напускането ѝ. Паметта на динамичните променливи се заделя от оператора new. Заделената по този начин памет остава свързана със съответната променлива докато не се освободи явно от програмиста. Явното освобождаване на динамична променлива се осъществява чрез оператора delete, приложен към указателя, който адресира съответната променлива. Едно непълно негово описание е дадено на Фиг. 14.9.

Оператор delete

Синтаксис

delete <указател_към_динамичен_обект>;

където <указател_към_динамичен_обект> е указател към динамичен обект (в широкия смисъл на думата), създаден чрез оператора new.

Семантика

Разрушава обекта, адресиран от указателя, като паметта, която заема този обект, се освобождава. Ако обектът, адресиран от указателя, е обект на клас, отначало се извиква деструкторът на класа (т.15.9) и след това се освобождава паметта.

Фиг. 14.9 Оператор delete

Ако в хийпа е заделена памет, след което тази памет не е освободена чрез delete, се получава загуба на памет. Парчето памет, което не е освободено, е като остров в хийпа, заемащо пространство, което иначе би могло да се използва за други цели.

За да се разруши масив, създаден чрез new по следния начин:

```
int* arr = new int[5];
```

трябва да се запише:

```
delete [] arr;
```

Ако обаче масивът съдържа в себе си указатели, първо трябва да бъде обходен и да бъде извикан операторът delete за всеки негов елемент. Едва след това може да се извика delete за масива по показания по-горе начин. Операторът delete трябва да се използва само за освобождаване на динамична памет, заделена с new. В противен случай действието му е непредсказуемо.

Динамичната памет не е неограничена. Тя може да се изчерпи по време на изпълнение на програмата. Неуспешното завършване на delete ускорява изчерпването. Ако наличната в момента динамична памет е недостатъчна, new връща нулев указател и програмата (функцията) няма да работи. Затова се препоръчва след всяко извикване на new да се прави проверка за успешността ѝ.

Чрез оператора new могат да се създават т. нар. **динамични масиви** – масиви с променлива дължина. Динамичните масиви се създават в динамичната памет. Следващата програма илюстрира този процес.

Задача 123. Да се напише програма, която създава динамичен масив от цели числа. Да се изведе масивът.

```
// Program 123. cpp
#include <iostream.h>
int main()
{int size; // дължина на масива
 do
 {cout << "size of array: ";
  cin >> size;
 } while (size<1);
 // създаване на динамичен масив arr от size елемента от
тип int
 int* arr = new int[size];
 int i;
 for (i = 0; i <= size-1; i++)
  arr[i] = i;
 // извеждане на елементите на arr
 for (i = 0; i <= size-1; i++)
  cout << arr[i] << " ";
 cout << endl;
 // освобождаване на заетата динамична памет
 delete [] arr;
 return 0;
}
```

Чрез оператора

```
delete [] arr;
```

е разрушен динамичният масив arr.

Методите на класовете също могат да използват динамична памет, която се заделя и освобождава по време на изпълнението им, чрез операторите new и delete.

Ще отбележим също, че заделената от член-функциите динамична памет не се освобождава автоматично при разрушаване на обектите на класове. Освобождаването на тази памет трябва да стане явно чрез оператора delete, който трябва да се изпълни преди разрушаването на обекта. Този процес може да бъде автоматизиран чрез използване на специален вид методи, наречени **деструктори**.

Деструктори

Разрушаването на обекти на класове в някои случаи е свързано с извършване на определени действия, които се наричат **заключителни**. Най-често тези действия са свързани с освобождаване на заделена преди това динамична памет, възстановяване на състояние на програмата и др. Ефектът от заключителните действия е противоположен на ефекта на инициализацията. Естествено е да се даде възможност заключителните действия да се извършат автоматично при разрушаването на обекта. Това се осъществява от деструкторите.

Деструкторът е член-функция, която се извиква при:
- разрушаването на обект чрез оператора delete,
- излизане от блок, в който е бил създаден обект от класа.

Един клас може да има явно дефиниран точно един деструктор. Името му съвпада с името на класа, предшествано от символа '~' (тилда), типът му е void и явно не се задава в заглавието. Деструкторът няма формални параметри.

Забележка: Използването на явно дефинирани деструктори не винаги е належащо, тъй като всички член-променливи се разрушават при разрушаването на обекта и без използването на деструктор. Ако конструкторът или някоя член-функция реализира динамично заделяне на памет за някоя член-данна, използването на деструктор е задължително, тъй като в този случай той трябва да освободи заетата памет.

Забележка: Ако се освобождава памет, заета от динамичен масив, чийто елементи са обекти на клас, трябва явно да се посочи дължината на масива. Тя е необходима за да се определи броят на извикванията на деструктора.

Създаване и разрушаване на обекти на класове

Съществуват два начина за създаване на обекти:

- чрез дефиниция;
- чрез функциите за динамично управление на паметта.

В първия случай обектът се създава при срещане на дефиницията (във функция или блок) и се разрушава при

завършване на изпълнението на функцията или при излизане от блока. Дефиницията може да бъде поставена където и да е в тялото на функцията или блока, като пред и след нея може да има изпълними оператори. Дефиницията, чрез която се създава обект, може да бъде допълнена с инициализация, която може да се основава на извикване на обикновен конструктор или на конструктора за присвояване.

Разрушаването на обекта е свързано с извикването на деструктора на класа, ако такъв явно е дефиниран или с извикването на "системния" деструктор (деструктора по подразбиране), ако в класа явно не е дефиниран деструктор.

Във втория случай създаването и разрушаването на обекти се управлява от програмиста. Създаването става с new, а разрушаването чрез delete. Операциите new се включват в конструкторите, а операциите delete – в деструктора на класа.

Пример:

```
rat *p = new rat(3,7); // търси в хийпа 8В, свързва адреса
                       // им с p, извиква конструктора
                       //  rat(9,13)  и инициализира паметта
(*p).print();
delete p;              //  извиква  деструктора,  след което
                       // разрушава обекта
...
```

В този случай деструкторът само регистрира присъствието си. Получаваме:

```
3/7
destruct number: 3/7
```

Оператори. Предефиниране на оператори

Езикът C++ има богат набор от оператори. В него са дадени също средства за предефиниране на оператори.

Всеки оператор се характеризира с:

- позиция на оператора спрямо аргументите му;

- приори тет;
- асоциативност.

Позицията на оператора спрямо аргументите му го определя като: префиксен (операторът е пред единствения му аргумент), инфиксен (операторът е между аргументите си) и постфиксен (операторът е след аргумента си).

Пример: Операторът / е инфиксен (4/8), операторът + е както инфиксен, така и префиксен (2+8, +78), а операторът ++ е както постфиксен, така и префиксен.

Приоритетът определя реда на изпълнение на операторите в операторен терм. Оператор с по-висок приоритет се изпълнява преди оператор с по-нисък приоритет.

Пример: Приоритетът на * и / е по-висок от този на + и -.

Асоциативността определя реда на изпълнение на оператори с еднакъв приоритет в операторен терм. В C++ има лявоасоциативни и дясноасоциативни оператори. Лявоасоциативните оператори се изпълняват отляво надясно, а дясноасоциативните – отдясно наляво.

В C++ не могат да се дефинират нови оператори, но всеки съществуващ едноаргументен или двуаргументен оператор с изключение на ::, ?:, ., *, # и ## може да бъде предефиниран от програмиста, стига поне един операнд на оператора да е обект на някакъв клас.

Например, възможно е да се предефинират операторите +, -, * и /, така че да могат да събират, изваждат, умножават и делят рационални числа. Тогава вместо sum(p, q), sub(p, q), mult(p, q) и quot(p, q) ще можем да пишем p+q, p-q, p*q и p/q, което безспорно е много по-удобно.

Предефинирането се осъществява чрез дефиниране на специален вид функции, наречени **операторни функции**. Последните имат синтаксис като на обикновените функции, но името им се състои от запазената дума operator, следвана от мнемоничното означение на предефинирания оператор. Когато предефинирането на оператор изисква достъп до компонентите на класове, обявени като private или protected, операторната дефиниция трябва да е член-функция или функция-приятел на

тези класове. Предефинираният оператор запазва всички характеристики на оригиналния.

Предефинирането може да стане по два начина:

- чрез функция-приятел;
- чрез член-функция.

Предефиниране чрез функция-приятел

В public частта на класа rat са включени декларациите на предефинираните оператори, предшествани от запазената дума friend:

```
friend rat operator+(rat const &, rat const &);
friend rat operator-(rat const &, rat const &);
friend rat operator*(rat const &, rat const &);
friend rat operator/(rat const &, rat const &);
```

Предефиниране чрез член-функция

В този случай първият аргумент на член-функцията трябва да е обект на класа и при дефинирането на операторната функция не се задава като параметър. Ако това не е така, операцията не може да се предефинира като член-функция.

Пример:

```
rat operator+(rat const &) const;
rat operator-(rat const &) const;
rat operator*(rat const &) const;
rat operator/(rat const &) const;
```

Наследяване. Производни класове.

Производните класове и наследяването са една от най-важните характеристики на обектно-ориентираното програмиране (ООП). Чрез механизма на наследяване от съществуващ клас се създава нов клас. Класът от който се създава се нарича **базов (основен) клас**, а този, който е създаден - **производен**. Понятията основен и производен клас са относителни, тъй като производен клас може да е основен за други класове, а основен – да е производен от други основни класове. Производният клас може да наследи компонентите на един или няколко базови класа. В първия случай наследяването се нарича **единично (просто)**, а във втория – **множествено**.

Дефинирането на производни класове е еквивалентно на конструирането на йерархии от класове. Ако множество от класове имат общи данни и методи, тези общи части могат да се обособят като основни класове, а всяка от останалите части да се дефинира като производен клас на съответния основен клас. Така се прави икономия на памет, тъй като се избягва многократното описание на едни и същи програмни фрагменти.

При конструирането на производни класове е достатъчно да се разполага само с обектните модули на основните класове, а не с техния програмен код. Това позволява да бъдат създавани библиотеки от класове, които да бъдат използвани при създаването на производни класове.

17.1 Дефиниране на производни класове

Подобно на обикновените, производните класове се дефинират като се *декларира класът* и се *дефинират неговите методи*.

Деклариране на производен клас

<декларация_на_производен_клас> ::=

class <име_на-производен_клас> :

[<атрибут_за_област>]$_{опц}$ <име_на_базов_клас>

{**,** [<атрибут_за_област>]$_{опц}$ <име_на_базов_клас>}$_{опц}$

{<декларация_на_компоненти>

};

<име_на-производен_клас> ::= <идентификатор>

<атрибут_за_област> ::= public | private | protected

<име_на_базов_клас> ::= <идентификатор>

Пред всяко име на базов клас *може* да се постави запазената дума public, private или protected. Нарича се **атрибут за област**, тъй като определя областта на наследените членове. Употребата на атрибутите за област е различна от тази за обявяване на секции в тялото на класа. Ако атрибут за област е пропуснат, подразбира се private. Атрибутът protected е включен в новите версии на езика и не се използва много често.

Примери:

1. Декларацията: class der : base1, base2, base3

{...

};

определя производен клас der с три основни класа base1, base2 и base3. Тъй като атрибутът за област е пропуснат и за трите базови класа, подразбира се private, т.е. декларацията е еквивалентна на

class der : private base1, private base2, private base3

{...

};

2. Декларацията: class der : public base1, base2, base3

{...

};

е еквивалентна на

class der : public base1, private base2, private base3

{...

};

3. Декларацията: class der : protected base1, base2, public base3

{...

};

е еквивалентна на

class der : protected base1, private base2, public base3

{...

};

Декларациите на компонентите на производен клас, а също дефинициите на неговите методи не се различават от съответните при обикновените класове.

Множеството от компонентите на един производен клас се състои от компонентите на неговите базови класове и компонентите, декларирани в самия производен клас. Оттук произлиза и терминът наследяване. Механизмът, чрез който производният клас получава компонентите на базовия, се нарича наследяване. Когато производният клас има няколко базови класа, той наследява компонентите на всеки от тях. Наследяването в този случай е множествено.

Процесът на наследяване се изразява в следното:

- наследяват се данните и методите на основния клас;
- получава се достъп до някои от наследените членове на основния клас;
- производният клас "познава" реализацията само на основния клас, от който произлиза;
- производният клас може да е основен за други класове.

Производният клас може да дефинира допълнително:

- свои член-данни;
- методи, аналогични на тези на основния клас, а също и нови.

Дефинираните в производния клас данни и методи се наричат **собствени**.

Производният клас е дефиниран след като вече е дефиниран базовият клас, от който той произлиза. Чрез него се разширява декларацията на съществуващ клас. Разширяемостта на класовете е една от важните характеристики на ООП.

17. 2 Наследяване и достъп до наследените компоненти

Ще напомним, че в рамките на един клас (без наследяване), protected частта има аналогична роля като тази на private частта. До компоненти от тип protected имат пряк достъп само член-функции и приятелски функции на класа.

Атрибутът за област на базовия клас в декларацията на производния клас (public, private или protected) управлява механизма на наследяване и определя какъв да бъде режимът на достъп до наследените членове. Таблицата от Фиг. 17.3 показва наследяванията на компоненти на основен клас в зависимост от атрибута за област.

Атрибут за област	Компонента на основен клас, определена като	Наследява се като
public	private public protected	Private public protected
Private	private public protected	Private private private
Protected	private public protected	Private protected protected

Фиг. 17.3 Наследявания на компоненти на основен клас в производен

От таблицата се вижда, че:

· Ако базовият клас е деклариран като public в производния клас, всички private, public и protected компоненти на базовия клас се наследяват съответно като private, public и protected компоненти на производния клас.

Пример: Ако

```
class base                                  class der1 : public base
{private: int b1;                          {private: int d1;
 protected: int b2;                         protected: int d2;
 public: int b3();                         public: int d3();
```

};

можем да си мислим, че der1 е клас от вида:

```
class der1
{private:
 int b1;
 int d1;
protected:
 int b2;
 int d2;
public:
 int b3();
 int d3();
};
```

· Ако базовият клас е деклариран като private в производния клас, всички негови компоненти се наследяват като private.

Пример: Ако

```
class base                          class der2 : private base
{private: int b1;                   {private: int d1;
 protected: int b2;                  protected: int d2;
 public: int b3();                   public: int d3();
};                                  };
```

можем да си мислим, че der2 е клас от вида:

```
    class der2
{private:
 int b1;
 int b2;
 int b3();
 int d1;
protected:
 int d2;
public:
 int d3();
};
```

· Ако базовият клас е деклариран като protected в производния клас, private компонентите му се наследяват като private, а public и protected – като protected.

Пример: Ако

```
class base                                    class der3 : protected base
{private: int b1;                             {private: int d1;
 protected: int b2;                            protected: int d2;
 public: int b3();                             public: int d3();
};                                                                  };
```

можем да си мислим, че der3 е клас от вида:

```
    class der3
{private:
```

```
 int b1;
 int d1;
protected:
 int b2;
 int d2;
 int b3();
public:
 int d3();
};
```

Наследените компоненти обаче се различават от декларираните в производния клас по правата за достъп. Производният клас има пряк достъп до компонентите, декларирани като public и protected, но няма пряк достъп до декларираните като private в базовия клас. Достъпът до private компонентите на базовия клас се извършва чрез неговия интерфейс.

Таблицата от Фиг. 17.4 показва прекия достъп на член-функции на производния клас (ПД) и външния достъп на производния клас (ВД) до компонентите на базовия клас.

компонента на базов клас	производен клас с атрибут public		производен клас с атрибут private		производен клас с атрибут protected	
	ПД	ВД	ПД	ВД	ПД	ВД
Public	да	да	да	не	да	не
Protected	да	не	да	не	да	не
Private	не	не	не	не	не	не

Фиг. 17.4 Достъп на член-функции и на обекти на производния клас до компонентите на базовия клас

Ще се спрем на някои от често срещаните случаи за достъп до членове на производен и основен клас, а също на достъпа на външни функции до наследен компонент. Ще изкажем и някои правила за достъп до компоненти на базови и производни класове, които ще подкрепим с още примери.

· Достъп до членове на основен клас чрез дефиниции на методи на производен клас

В сила са следните правила за достъп:

- Методите на производен клас (без значение на атрибута за област) нямат директен достъп до членовете от private–секцията на основния му клас

- В дефинициите на собствени методи на производния клас могат да се използват методите от секциите public и protected на основния му клас

- В дефинициите на собствени методи на производния клас може директно да се използват член-данните на секцията protected на основния му клас

· Достъп до методи чрез обекти на основния и производния клас

Обект на основен клас има пряк достъп до всички свои компоненти, обявени като public и няма пряк достъп до компонентите, обявени като private и protected. Обект на производен клас има пряк достъп до public компонентите на собствения си и компонентите на основния клас, наследени в производния клас като public.

· Достъп на основен клас до членове на производен клас и обратно

Методите на основния клас нямат достъп до членове на производен клас. Причината е, че когато основният клас се дефинира, не е ясно какви производни класове ще произхождат от него.

Производният клас също няма привилигирован достъп до членове на основния клас. От фиг. 17.3 се вижда, че производните класове нямат достъп до методите, обявени като private в основния клас.

Допустими са редица присвоявания между обекти на основния и производния клас. Ще ги разгледаме подробно в следващи части на главата. Засега ще отбележим само, че за реализирането им се извършват редица преобразувания.

· Достъп на функции приятели на производен клас до компоненти на основния му клас

Ще напомним, че функциите-приятели на клас не са елементи на класа, на който са приятели. Те са външни функции, получили привилигировън достъп до компонентите на класа.

Функциите приятели на производен клас имат същите права на достъп като член-функциите на производния клас. *Имат пряк достъп до всички компоненти, декларирани в класа и до public и protected компонентите на основния клас. Декларацията за приятелство не се наследява. Функция приятел на базовия клас не е приятел (освен ако не е декларирана като такава) на производния клас*.

11. Обектно ориентирано програмиране (на базата на C++ или Java): Основни принципи. Класове и обекти. Конструктори и деструктори. Оператори. Производни класове и наследяване.

Основните характеристики на обектно ориентираното програмиране са:

- Всичко е обект
- Една програма е съвкупснот от обекти, указващи си един на друг чрез изпраяане на съобщения какво да правят
- Обектите притежават собствена памет, суставена от други обекти.
- Всеки обект има тип
- Всички обекти от определен тип могат да получават едни и същи съобщения.

Обектно-ориентираното програмиране се е наложило като стандарт при почти всички съвременни езици за програмиране. То предоставя мощно средство за моделиране на обектите от реалния свят и взаимоотношенията между тях, позволява добро структуриране на програмния код и улеснява неговото преизползване. Благодарение на капсулацията на данните, както и на възможностите за наследяване и работа с абстрактни данни операции, ООП се е утвърдило като предпочитан подход при създаване на големи приложения и библиотеки.

Четирите основни принципа на ООП са капсулация на данните, абстракция, наследяване и полиморфизъм. Те са основните характеристики, които определят един език за програмиране като обектно-ориентиран.

Капсулация на данните

Основна концепция в ООП е обектът да се разглежда като "черна кутия" – използващите обекта "виждат" само атрибутите и операциите, които са присъщи на обекта от реалния свят, без да се интересуват от конкретната им реализация – клиентът на обекта трябва да знае само какво може да прави обектът, а не как го прави. В такъв смисъл "капсулация" означава скриване на ненужните детайли за обектите и откриване към външния свят само на важните техни характеристики и свойства. Обектите в ООП съдържат своите данни и средствата за тяхната обработката, капсулирани като едно цяло.

Абстракция

Абстракцията опростява сложната реалност чрез моделиране на класове подходящи за разрешаването на определен проблем. Тя представлява модел, изглед или някакво друго общо представяне на действителен обект. Абстракцията е разработване на програмен обект представящ обект от реалния свят, а капсулацията му скрива детайлите на имплеметацията. Абстракцията на данните налага ясно разделяне между абстрактните свойства на съответния тип и конкретните детайли на представянето. Идеята е, че промени в това представяне не трябва да имат влияние върху код използващ абстракцията. Нека разгледаме един прост пример: имаме клас Person съответно със ссвойства Height, Weight, Age и т.н. и действия, които може да се извършват с него: Run(), Sleep(), Cry(). Абстрактното представяне на Person се определя от свойствата които притежава и от действията, които могат да се извършват върху него и от информацията, която може да получим или предадем на него. Конкретната имплементация на действията и свойствата на обекта остава скрита от външния свят, който няма достъп до нея.

Наследяване

Ако един обект съдържа всички свойства и действия на друг, първият може да го наследи. По този начин наследеният обект освен собствените си атрибути и операции приема и тези на "родителя" си (базовия клас), като така се избягва повторното им дефиниране и се позволява създаването на йерархии от класове, моделиращи по естествен начин зависимостите от реалността.
За да изясним това понятие, ще си послужим с класическият в ООП пример за класа от обекти Animal, който представлява абстракция за множеството от всички животни. Всички обекти от този клас имат общи характеристики (например цвят и възраст) и обща функционалност, например операциите Eat и Sleep, докато за класът Dog, представляващ множеството от всички кучета, които също са животни, би могъл да предоставя операциите Eat, Sleep и Bark. Удачно е класът Dog да наследи Animal – тогава той ще съдържа описание само на собственото си действие Bark, докато тези на Eat и Sleep ще получи от базовия си клас. Чрез наследяването се постига специализация, или конкретизация на класовете, тъй като базовият клас представлява категория от обекти по-обща от тази на наследяващите го. Ако си послужим с горния пример, множествата на кучетата и котките са подмножества на множеството от всички животни.

Полиморфизъм

Полиморфизъм буквално означава приемането на различни форми от един обект. Нека е даден базов клас, представящ категория от обекти, които реализират общо действие, което се наследява от множество класове, описващи по-тесни категории. Въпреки, че те всички споделят това действие, те могат да го реализират по различен начин. Когато разполагаме с обекти от базовия клас, знаем че всички те реализират това действие, независимо на кой наследен клас принадлежат. Поради това можем да го използваме без да се интересуваме от конкретната му реализация. Например, ако класът Animal предоставя действието Talk и разгледаме наследяващите го класове Dog и Cat, всеки от тях го реализира по конкрете начин. И ако имаме животно, което издава звук и то е куче – ще лае, а ако е котка – ще мяучи. Полиморфизмът позволява унифицираното извършване на действие над различни обекти, които го реализират. В този случай издаването на звук от животно е полиморфно действие – такова, което се реализира по различен начин в различните наследници на базовия клас.

Класове

Класовете са нови типове данни, дефинирани от потребителя. Те могат да обогатяват възможностите на вече съществуващ тип или да представят напълно нов тип данни. Класовете са подобни на структурите, даже може да се каже, че в някои отношения са почти идентични. В C++ класът може да се разглежда като структура, на която са наложени някои ограничения по отношение на правата на достъп. Отначало ще разгледаме основното, което е приложимо както за класовете, така и за структурите. Основен принцип на процедурното програмиране е модулния. Програмата се разделя на “подходящи” взаимносвързани части (функции, модули), всяка от които се реализира чрез определени средства. Важен е обаче начинът, по който да става определянето на частите и връзките помежду им. Целта е, следващи промени в представянето на данните да не променят голям брой от модулите на програмата. Разсъждения в тази посока довеждат до подхода абстракция със структури от данни. При него методите за използване на данните се разделят от методите за тяхното конкретно представяне.

Програмите се конструират така, че да работят с “абстрактни данни” – данни с неуточнено представяне.

Дефиниране на класове

Класовете осигуряват механизми за създаване на напълно нови типове данни, които могат да бъдат интегрирани в езика, а също за обогатяване възможностите на вече съществуващи типове. Дефинирането на един клас се състои от две части:
декларация на класа и
дефиниция на неговите член-функции (методи).

Декларация на клас

Декларацията на клас се състои от заглавие и тяло. Заглавието започва със запазената дума class, следвано от името на класа. Тялото е заградено във фигурни скоби. След скобите стои знакът “;” или списък от обекти. В тялото на класа са декларирани членовете на класа (член-данни и член-функции) със съответните им нива на достъп.
За имената на класовете важат същите правила, които се прилагат за имената на всички останали типове и променливи. Също като при структурите името на класа може да бъде пропуснато.
Имената на членовете на класа са локални за него, т.е. в различни класове в рамките на една програма могат да се дефинират членове с еднакви имена. Член-данни от един и същ тип могат да се изредят, разделени със запетая и предшествани от типа им.

Пример:

```
class point
{private:
  double x, y;              // x и y са член-данни
 public:
  point(double, double);
  void read();
  int get_x() const;
  int get_y() const;
  void print() const;
}p=point(2,7), q(-2,3), r=q;
```

В тялото, някои декларации на членове могат да бъдат предшествани от спецификаторите на достъп private, public или protected. Областта на един спецификатор на достъп започва от спецификатора и продължава до следващия спецификатор. Подразбиращ се спецификатор за достъп е private. Един и същ спецификатор на достъп може да се използва повече от веднъж в декларация на клас.
Препоръчва се, ако секция public съществува, да бъде първа в декларацията, а секцията private да бъде последна в тялото на класа.

Достъпът до членовете на класовете може да се разгледа на следните две нива:
- По отношение на член-функциите в класа е в сила, че те имат достъп до всички членове на класа. При това не е необходимо тези компоненти да се предават като параметри. Този режим на достъп се нарича режим на пряк достъп.
- По отношение на функциите, които са външни за класа, режимът на достъп са определя от начина на деклариране на членовете.

Членовете на даден клас, декларирани като private (декларирани след запазената дума private) са видими (достъпни) само в рамките на класа. Външните функции нямат достъп до тях. По подразбиране членовете на класовете са private. Чрез използването на членове, обявени като private, се постига скриване на членове за външната за класа среда. Процесът на скриване се нарича още капсулиране на информацията.
Членовете на клас, които трябва да бъдат видими извън класа (да бъдат достъпни за функции, които не са методи на дадения клас) трябва да бъдат декларирани като public (декларирани след запазената дума public).
Дефиниране на методите на клас

След декларирането на клас, трябва да се дефинират неговите методи. Дефинициите са аналогични на дефинициите на функции, но името на метода се предшества от името на класа, на който принадлежи метода, следвано от оператора за принадлежност :: (Нарича се още оператор за област на действие). Такива имена се наричат пълни.

Дефиниция на метод на клас

Ще отбележим, че дефиницията на конструктор не започва с <тип>, а запазената дума const може да присъства само в дефинициите на функциите за достъп. Добрият стил на програмиране изисква използването на const в дефинициите на функциите за достъп и също в техните декларации.Ако се пренебрегне това изискване, могат да се създадат класове, които да не могат да се използват от други програмисти.
Обикновено дефинициите на методите са разположени веднага след декларирането на класа, на който те са членове. Възможно е обаче, дефинициите на методите на един клас да бъдат част от декларациите на този клас, т.е. в декларациите на член-функциите на класа могат да се зададат не само прототипите им, но и техните тела.

Пример: Класът rat може да бъде дефиниран и по следния начин:

```
class rat
{private:
 int numer;
 int denom;
public:
 rat()
 {numer = 0;
  denom = 1;
 }
 rat(int a, int b)
 {if (a == 0 || b==0)
 {numer = 0;
```

```
 denom = 1;
}
 else
 {int g = gcd(abs(a), abs(b));
  if (a>0 && b>0 || a<0 && b<0)
        {numer = abs(a)/g;
         denom = abs(b)/g;}
  else
  {numer = - abs(a)/g;
   denom = abs(b)/g;
  }
  }
 }
void read()
{cout << "numer: ";
 cin >> numer;
 do
 {cout << "denom: ";
  cin >> denom;
 } while (denom == 0);
}
int get_numer() const
{return numer;
}
int get_denom() const
{return denom;
}
void print() const
{cout << numer << "/" << denom << endl;
}
};
```

В този случай обаче член-функциите се третират като вградени (inline) функции.
Често член-функциите се реализират като вградени функции. Това увеличава ефективността на програмата, използваща класа.

Ще отбележим също, че в тялото на дефиницията на член-функция явно не се указва обектът, върху който тя ще се приложи. Този обект участва неявно - чрез член-данните на класа. Заради това се нарича неявен параметър, а член-данните – абстрактни данни.Параметри, които участват явно в дефиницията на член-функция се наричат явни. Всяка член-функция има точно един неявен параметър и нула или повече явни.

Област на класовете

За разлика от функциите, класовете могат да се декларират на различни нива в програмата: глобално (ниво функция) и локално (вътре във функция или в тялото на клас).
Областта на глобално деклариран клас започва от декларацията и продължава до края на програмата. Примерите досега бяха с такива класове.
Ако клас е деклариран във функция, всички негови член-функции трябва да са вградени (inline). В противен случай ще се получат функции, дефинирани във функция, което не е възможно.

Пример:

```
void f(int i, int* p)
{int k;
 class CL
 {public:
        // всички методи са дефинирани в тялото на класа
  ...
  private:
  ...
 };
// тяло на функцията f
 CL x;
 ...
}
```

Областта на клас, дефиниран във функция, е функцията. Обектите на такъв клас са видими само в тялото на функцията.
Възможно е използването на обекти (в широкия смисъл на думата) с еднакви имена. В сила е правилото, че в областта си локалният обект скрива нелокалния.
Не е възможно в тялото на локално дефиниран клас да се използва функцията, в която класът е дефиниран.

Обекти

След като даден клас е дефиниран, могат да бъдат създавани негови екземпляри, които се наричат обекти. Връзката между клас и обект в езика C++ е подобна на връзката между тип данни и променлива, но за разлика от обикновените променливи, обектите се състоят от множество компоненти (член-данни и член-функции). На Фиг. 14.3 е даден синтаксисът на дефиниция на обект на клас.

Декларацията на клас не заделя памет за него. Памет се заделя едва при дефинирането на обект от класа. Дефиницията

```
rat p, q(2, 3), r = rat(3, 8);
```

заделя за обектите p, q и r по 8 байта ОП (по 4B за всяка от данните им numer и denom).

Достъпът до компонентите на обектите (ако е възможен) се осъществява чрез задаване на името на обекта и името на данната или метода, разделени с точка. Изключение от това правило правят конструкторите.

Пример:

```
rat p(1,2), q;
p.get_numer() // достъп до член-функцията get_numer() за обекта p
q.get_numer()          // достъп до член-функцията get_numer() за обекта q
```

Ще отбележим също, че на практика обектите p и q нямат свои копия на метода get_numer(). И двете обръщения се отнасят за един и същ метод, но при първото обръщение се работи с данните за обекта p, а при второто – с данните за обекта q.
При създаването на обекти на един клас кодът на методите на този клас не се копира във всеки обект, а се намира само на едно място в паметта.
Естествено възниква въпросът по какъв начин методите на един клас "разбират" за кой обект на този клас са били извикани. Отговорът на този въпрос дава указателят this. Всяка член-функция на клас поддържа допълнителен формален параметър - указател с име this и от тип <име_на_клас>*.
Обекти от един и същ клас могат да се присвояват един на друг. Присвояването може да е и на ниво инициализация .

Пример: Допустими са дефинициите

```
rat p, q(4,5), r=q;
p = q;
...
r = p;
```

При присвояването се копират всички член-данни на обекта. Така присвояването

```
r = p;
```

е еквивалентно на

```
r.numer = p.numer;
r.denom = p.denom;
```

Конструктори

Създаването на обекти е свързано с отделяне на памет, запомняне на текущо състояние, задаване на начални стойности и др. дейности, които се наричат инициализация на обекта. В езика C++ тези дейности се изпълняват от специален вид член-функции на класовете – конструкторите.

Дефиниране на конструктор

Ще напомним, че конструкторът е член-функция, която притежава повечето характеристики на другите член-функции, но има и редица особености, като:
Името на конструктора съвпада с името на класа.
Типът на резултата е указателят this и явно не се указва.
Изпълнява се автоматично при създаване на обекти.
Не може да се извиква явно (обръщение от вида r.rat(1,4) е недопустимо).

Освен това в един клас може явно да не е дефиниран конструктор, но може да са дефинирани и няколко конструктора.
Типична дефиниция на конструктор е дадена в следващия пример.

Пример:

```
class CL
{public:
  CL(int, int, int);
  void print();
  ...
 private:
  int a, b, c;
};
CL::CL(int x, int y, int z) // конструктор с три параметъра
{a = x;
 b = y;
 c = z;
}
```

Класът CL в този пример има един триаргументен конструктор. При създаване на обект на класа, този конструктор ще се изпълнява автоматично, стига да е коректно извикан, в резултат на което член-данните a, b и c на създадения обект ще се свържат със стойностите, които се подадат като фактически параметри на конструктора.

Забелязваме, че е възможно член_данна да се свърже с инициализираща стойност в заглавието на конструктора.

Пример: Конструкторът на класа CL може да се дефинира и по следния начин

```
CL::CL(int x, int y, int z): a(x), b(y), c(z)
{}
```

Ще отбележим, че не е задължително всички член-данни да са инициализирани само пред тялото или само вътре в тялото на конструктора.

Пример: Допустима е дефиницията

```
CL::CL(int x, int y, int z): a(x)
    {b = y;
     c = z;
    }
```

Смисълът на обобщената синтактична конструкция е, че инициализацията на член-данните в заглавието предшества изпълнението на тялото на конструктора. Това я прави изключително полезна. Използването й увеличава ефективността на класа.

Предефинирани конструктори

Обект (в общия смисъл) е предефиниран, ако за него има няколко различни дефиниции, задаващи различни негови интерпретации. За да бъдат използвани такива конструкции е необходим критерии, по който те да се различат.
В рамките на една програма може да се извършва предефиниране на функции. Възможно е:

а) да се използват функции с едно и също име с различни области на видимост
В този случай не възниква проблем с различаването.
б) да се използват функции с едно и също име в една и съща област на видимост
В един клас може да са дефинирани няколко конструктора. Всички те имат едно име (името на класа), но трябва да се различават по броя и/или типа на параметрите си. Наричат се предефинирани конструктори. При създаването на обект на класа се изпълнява само един от тях. Определя се съгласно критерия за най-добро съвпадане.
Пример: В класа rat дефинирахме два конструктора rat() и rat(int, int), които се различават по броя на параметрите си.

Подразбиращ се конструктор

В клас може явно да е дефиниран, но може и да не е дефиниран конструктор. Ако явно не е дефиниран конструктор, автоматично се създава един т. нар. подразбиращ се конструктор. Този конструктор реализира множество от действия като: заделяне на памет за данните на обект, инициализиране на някои системни променливи и др.

Подразбиращият се конструктор може да бъде предефиниран. За целта е необходимо в класа да бъде дефиниран конструктор без параметри.

Пример: В класа rat, подразбиращият се конструктор беше предефиниран от конструктора

```
rat::rat()
{numer = 0;
 denom = 1;
}
```

Конструктори с подразбиращи се параметри

Функциите в езика C++ могат да имат подразбиращи се параметри. За тези параметри се задават подразбиращи се стойности, които се използват само ако при извикването на функцията не бъде зададена стойност за съответния параметър.
Задаването на подразбираща се стойност се извършва чрез задаване на конкретна стойност в прототипа на функцията или в нейната дефиниция.
При използване на подразбиращи се параметри, важна роля играе редът на параметрите. Прието е, че ако параметър на функция е подразбиращ се, всички параметри след него също са подразбиращи се.

Всичко казано за подразбиращите се параметри на функции се отнася и за конструкторите.
Ако променим дефиницията на класа rat по следния начин:

```
class rat
{private:
  int numer;
  int denom;
 public:
  rat(int=0, int=1);
  void read();
```

```
    int get_numer() const;
    int get_denom() const;
    void print() const;
};
```

където конструкторът rat(int, int); се дефинира по същия начин, са допустими следните дефиниции на обекти:

```
rat p,                          // p се инициализира с 0/1
      q = rat(),                // q се инициализира с 0/1
      r = rat(5),               // r се инициализира с 5/1
      s = rat(13,21),           // s се инициализира с 13/21
      t(2);                     // t се инициализира с 2/1
```

Конструктори за присвояване и копиране

Инициализацията на новосъздаден обект на даден клас може да зависи от друг обект на същия клас. За да се укаже такава зависимост се използва знакът за присвояване или кръгли скоби.
Пример: Нека
rat p = rat(1,4);
Чрез еквивалентните конструкции

```
rat q = p;
rat q(p);
```

се създава обектът q от клас rat, като инициализацията на q зависи от p. Тази инициализация се създава от специален конструктор, наречен конструктор за присвояване.
Конструкторът за присвояване е конструктор, поддържащ формален параметър от тип <име_на_клас> const &.
- Ако в един клас явно не е дефиниран конструктор за присвояване, компилаторът автоматично създава такъв, в момента когато новосъздаден обект се инициализира с обекта, намиращ се от дясната страна на знака за присвояване или в кръглите скоби. Този конструктор за присвояване се нарича конструктор за копиране.

Дефиницията rat p=q създава нов обект p (без викане на конструктор), в който се копират съответните стойности на обекта q. Това е резултат от изпълнението на обръщението към rat(&p, q).
- Ако в класа е дефиниран конструктор за присвояване, компилаторът го използва.

Предефиниране на служебния конструктор за копиране се налага когато обектите имат член-данни, указващи динамична памет.

Освен в горните случаи, конструкторът за присвояване (копиране) се използва и при предаване на обект като аргумент на функция, когато предаването е по стойност, а също при връщане на обект като резултат от изпълнение на функция.

Указатели към обекти на класове

Дефинират се по същия начин както се дефинират указатели към променливи от стандартните типове данни.

Пример:

rat p;

rat * ptr = &p;

В резултат ще се отделят 4В ОП, които ще се именуват с ptr и ще се инициализират с адреса на обекта p.

Достъпът до компонентите на рационалното число, сочено от ptr, се осъществява по общоприетия начин:

(*ptr).get_numer()

(*ptr).get_denom()

Синтактичната конструкция (*ptr). е еквивалентна на ptr ->. Така горните обръщения могат да се запишат и по следния начин:

ptr -> get_numer()

ptr -> get_denom()

Масиви и обекти

Елементите на масив могат да са обекти, но разбира се от един и същ клас. Дефинират се по общоприетия начин (Фиг. 14.7).

Пример:

rat table[10];

определя масив от 10 обекта от клас rat.

Достъпът до елементите на масива е пряк и се осъществява по стандартния начин – чрез индексирани променливи.

Пример: Чрез индексираните променливи

table[0], table[1], ..., table[9]

се осъществява достъп до първия, втория и т.н. до десетия елемент на table.

Тъй като table[i] (i = 0, 1, ..., 9) са обекти, възможни са следните обръщения към техни компоненти:

table[i].read(); // въвежда стойност на table[i]

table[i].print(); // извежда стойността на table[i]

table[i].get_numer(); // намира числителя на table[i]

table[i].get_denom(); // намира знаменателя на table[i].

Връзката между масиви и указатели е в сила и в случая когато елементите на масива са обекти. Името на масива е указател към първия му елемент, т.е. ако

```
rat * p = table;          // p сочи към table[0]
                                                    // т.е. p==&table[0]
*(p+i) == table[i], i = 0, 1,...,9
```

Тогава

```
(*(p+i)).print();                  // е еквивалентно на table[i].print();
```

Масивът може да е член-данна на клас.
Конструкторите (в частност конструкторът по подразбиране) играят важна роля при дефинирането и инициализирането на масиви от обекти. Масив от обекти, дефиниран в програма, се инициализира по два начина:
неявно (чрез извикване на системния конструктор по подразбиране за всеки обект – елемент на масива);
явно (чрез инициализиращ списък).

Примери:

а) Класът

```
const NUM = 5;
class student
{public:
 void read_student();
 void print_student() const;
 bool is_better(student const &) const;
 double average() const;
 private:
 int facnom;
 char name[26];
 double marks[NUM];
```

```
};
```

няма явно дефиниран конструктор. Дефиницията
student table[30];
на масива table от 30 обекта от клас student е правилна.
Инициализацията се осъществява чрез извикване на "системния" конструктор по подразбиране за всеки обект – елемент на масива.
б) Класът rat, дефиниран по-долу

```
class rat

{private:

 int numer;

 int denom;

public:

 rat(int=0, int=1);

 void read();

 int get_numer() const;

 int get_denom() const;

 void print() const;

};
```

притежава явно дефиниран конструктор с два подразбиращи се параметъра. В този случай са допустими дефиниции от вида:

```
rat x[10]; // x[i] се инициализира с 0/1, за всяко i=0,1,...9.

rat x[10] = {1,2,3,4,5,6,7,8,9,10}; //x[i] == i/1

rat x[10] = {rat(1,21),rat(2),rat(3, 5),4,5,6,7,8,9,10};

// x[0] == 1/21; x[1] == 2/1; x[2] == 3/5, x[3]== 4/1, ...
```

Ако променим конструктора на класа rat от

```
rat(int=0, int=1);
```

в

```
rat(int, int);
```

т.е. без подразбиращи се параметри и трите дефиниции от по-горе ще съобщят за грешка. Единствено допустима дефиниция на x[10] е с инициализация с 10 обръщения към двуаргументния конструктор rat с явно указани два аргумента.

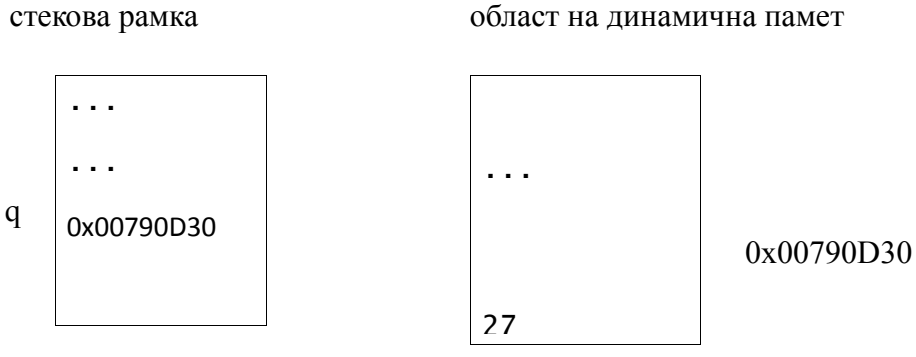
Динамични обекти

Вече разгледахме в най-общ план разпределението на ОП по време на изпълнението на програма. От Фиг. 8.1 на Глава 8 се вижда, че всяка програма има две "места" за памет: програмен стек (стек) и област за динамичните данни (динамична памет, хийп или куп). Стекът е област за временно съхранение на информация. Той е кратковременна памет. C++ използва стека основно за реализиране на обръщения към функции. Всяко обръщение към функция предизвиква конструиране на нова стекова рамка, която се установява на върха на стека. По такъв начин когато функция A извика функция B, която от своя страна вика функция C, стекът нараства. Когато пък всяка от тези функции завършва, стековите рамки на тези функции автоматично се разрушава. Така стекът се свива.
Хийпът е по-постоянна област за съхранение на данни. Той е един вид дълготрайна памет. Особеност на тази памет е, че тя не се свързва с имена на променливи. С разположените в нея обекти се работи косвено – чрез указатели. Обикновено се използва при работа с т. нар. динамични структури от данни. Динамичните данни са такива обекти (в широкия смисъл на думата), чийто брой не е известен в момента на проектирането на програмата. Те се създават и разрушават по време на изпълнението на програмата. След разрушаването им, заетата от тях памет се освобождава и може да се използва отново. Така паметта се използва по-ефективно. Създаването и разрушаването на динамични обекти в C++ се осъществява чрез операторите new и delete. Извикването на new заделя в хийпа необходимата памет и връща указател към нея. Този указател може да се съхрани в някаква променлива и да се пази докато е необходимо. За разлика от стека, заделянето на памет в хийпа е явно – чрез new. Освобождаването на паметта от хийпа също става явно, чрез delete. Всяко извикване на new трябва да бъде балансирано чрез извикване на delete. Последното се налага, тъй като за разлика от стека, хийпът не се изчиства автоматично. В C++ няма система за "събиране на боклуци" (автоматично премахване на обекти, които вече не са необходими). Затова трябва явно да бъдат изтрити създадените в хийпа обекти.

Примери:

a) int* q = new int(2+5*5);

отделя (ако е възможно) 4B памет в хийпа, инициализира я с 27 - стойността на израза 2+5*5 и свързва q с адреса на тази памет, т.е.



Заделянето на памет по време на компилация се нарича статично заделяне на памет, заделянето на памет по време на изпълнение на програмата - динамично разпределение на паметта.

Под период на активност на една променлива се разбира частта от времето за изпълнение на програмата, през което променливата е свързана с конкретно място в паметта. Паметта за глобалните променливи се заделя в началото и остава свързана с тях до завършването на изпълнението на програмата. Паметта за локалните променливи се заделя при влизане в локалната област и се освобождава при напускането й. Паметта на динамичните променливи се заделя от оператора new. Заделената по този начин памет остава свързана със съответната променлива докато не се освободи явно от програмиста. Явното освобождаване на динамична променлива се осъществява чрез оператора delete, приложен към указателя, който адресира съответната променлива.

Ако в хийпа е заделена памет, след което тази памет не е освободена чрез delete, се получава загуба на памет. Парчето памет, което не е освободено, е като остров в хийпа, заемащо пространство, което иначе би могло да се използва за други цели.

За да се разруши масив, създаден чрез new по следния начин:

```
int* arr = new int[5];
```

трябва да се запише:

```
delete [] arr;
```

Ако обаче масивът съдържа в себе си указатели, първо трябва да бъде обходен и да бъде извикан операторът delete за всеки негов елемент. Едва след това може да се извика delete за масива по показания по-горе начин. Операторът delete трябва да се използва само за освобождаване на динамична памет, заделена с new. В противен случай действието му е непредсказуемо.

Динамичната памет не е неограничена. Тя може да се изчерпи по време на изпълнение на програмата. Неуспешното завършване на delete ускорява изчерпването. Ако наличната в момента динамична памет е недостатъчна, new връща нулев указател и програмата (функцията) няма да работи. Затова се препоръчва след всяко извикване на new да се прави проверка за успешността й.

Чрез оператора new могат да се създават т. нар. динамични масиви – масиви с променлива дължина. Динамичните масиви се създават в динамичната памет. Следващата програма илюстрира този процес.

(за примерни задачи вж. учебника)

Методите на класовете също могат да използват динамична памет, която се заделя и освобождава по време на изпълнението им, чрез операторите new и delete.

Ще отбележим също, че заделената от член-функциите динамична памет не се освобождава автоматично при разрушаване на обектите на класове. Освобождаването на тази памет трябва да стане явно чрез оператора delete, който трябва да се изпълни преди разрушаването на обекта. Този процес може да бъде автоматизиран чрез използване на специален вид методи, наречени деструктори.

Деструктори

Разрушаването на обекти на класове в някои случаи е свързано с извършване на определени действия, които се наричат заключителни. Най-често тези действия са свързани с освобождаване на заделена преди това динамична памет, възстановяване на състояние на програмата и др. Ефектът от заключителните действия е противоположен на ефекта на инициализацията. Естествено е да се даде възможност заключителните действия да се извършат автоматично при разрушаването на обекта. Това се осъществява от деструкторите.

Деструкторът е член-функция, която се извиква при:

- разрушаването на обект чрез оператора delete,
- излизане от блок, в който е бил създаден обект от класа.

Един клас може да има явно дефиниран точно един деструктор. Името му съвпада с името на класа, предшествано от символа ‘~’ (тилда), типът му е void и явно не се задава в заглавието. Деструкторът няма формални параметри.

Забележка: Използването на явно дефинирани деструктори не винаги е належащо, тъй като всички член-променливи се разрушават при разрушаването на обекта и без използването на деструктор. Ако конструкторът или някоя член-функция реализира динамично заделяне на памет за някоя член-данна, използването на деструктор е задължително, тъй като в този случай той трябва да освободи заетата памет.
Забележка: Ако се освобождава памет, заета от динамичен масив, чийто елементи са обекти на клас, трябва явно да се посочи дължината на масива. Тя е необходима за да се определи броят на извикванията на деструктора.

Създаване и разрушаване на обекти на класове

Съществуват два начина за създаване на обекти:
чрез дефиниция;
чрез функциите за динамично управление на паметта.
В първия случай обектът се създава при срещане на дефиницията (във функция или блок) и се разрушава при завършване на изпълнението на функцията или при излизане от блока. Дефиницията може да бъде поставена където и да е в тялото на функцията или блока, като пред и след нея може да има изпълними оператори. Дефиницията, чрез която се създава обект, може да бъде допълнена с инициализация, която може да се основава на извикване на обикновен конструктор или на конструктора за присвояване.
Разрушаването на обекта е свързано с извикването на деструктора на класа, ако такъв явно е дефиниран или с извикването на "системния" деструктор (деструктора по подразбиране), ако в класа явно не е дефиниран деструктор.
Във втория случай създаването и разрушаването на обекти се управлява от програмиста. Създаването става с new, а разрушаването чрез delete. Операциите new се включват в конструкторите, а операциите delete – в деструктора на класа.

Пример:

```
rat *p = new rat(3,7); // търси в хийпа 8B, свързва адреса
                                                    // им с p, извиква конструктора
                                                    // rat(9,13) и инициализира
паметта
(*p).print();
delete p;                                   // извиква деструктора, след което
                                            // разрушава обекта
...
```

В този случай деструкторът само регистрира присъствието си. Получаваме:

```
3/7
destruct number: 3/7
```

Оператори. Предефиниране на оператори

Езикът C++ има богат набор от оператори. В него са дадени също средства за предефиниране на оператори.

Всеки оператор се характеризира с:
позиция на оператора спрямо аргументите му;
приоритет;
асоциативност.

Позицията на оператора спрямо аргументите му го определя като: префиксен (операторът е пред единствения му аргумент), инфиксен (операторът е между аргументите си) и постфиксен (операторът е след аргумента си).
Пример: Операторът / е инфиксен (4/8), операторът + е както инфиксен, така и префиксен (2+8, +78), а операторът ++ е както постфиксен, така и префиксен.
Приоритетът определя реда на изпълнение на операторите в операторен терм. Оператор с по-висок приоритет се изпълнява преди оператор с по-нисък приоритет.
Пример: Приоритетът на * и / е по-висок от този на + и -.
Асоциативността определя реда на изпълнение на оператори с еднакъв приоритет в операторен терм. В C++ има лявоасоциативни и дясноасоциативни оператори. Лявоасоциативните оператори се изпълняват отляво надясно, а дясноасоциативните – отдясно наляво.
В C++ не могат да се дефинират нови оператори, но всеки съществуващ едноаргументен или двуаргументен оператор с изключение на ::, ?:, ., *, # и ## може да бъде предефиниран от програмиста, стига поне един операнд на оператора да е обект на някакъв клас.
Например, възможно е да се предефинират операторите +, -, * и /, така че да могат да събират, изваждат, умножават и делят рационални числа. Тогава вместо sum(p, q), sub(p, q), mult(p, q) и quot(p, q) ще можем да пишем p+q, p-q, p*q и p/q, което безспорно е много по-удобно.
Предефинирането се осъществява чрез дефиниране на специален вид функции, наречени операторни функции. Последните имат синтаксис като на обикновените функции, но името им се състои от запазената дума operator, следвана от мнемоничното означение на предефинирания оператор. Когато предефинирането на оператор изисква достъп до компонентите на класове, обявени като private или protected, операторната дефиниция трябва да е член-функция или функция-приятел на тези класове. Предефинираният оператор запазва всички характеристики на оригиналния.
Предефинирането може да стане по два начина:
чрез функция–приятел;
чрез член-функция.

Предефиниране чрез функция-приятел

В public частта на класа rat са включени декларациите на предефинираните оператори, предшествани от запазената дума friend:

```
friend rat operator+(rat const &, rat const &);
friend rat operator-(rat const &, rat const &);
friend rat operator*(rat const &, rat const &);
friend rat operator/(rat const &, rat const &);
```

Предефиниране чрез член-функция

В този случай първият аргумент на член-функцията трябва да е обект на класа и при дефинирането на операторната функция не се задава като параметър. Ако това не е така, операцията не може да се предефинира като член-функция.

Пример:

```
rat operator+(rat const &) const;
rat operator-(rat const &) const;
rat operator*(rat const &) const;
rat operator/(rat const &) const;
```

Наследяване. Производни класове.

Производните класове и наследяването са една от най-важните характеристики на обектно-ориентираното програмиране (ООП). Чрез механизма на наследяване от съществуващ клас се създава нов клас. Класът от който се създава се нарича **базов (основен) клас**, а този, който е създаден - **производен**. Понятията основен и производен клас са относителни, тъй като производен клас може да е основен за други класове, а основен – да е производен от други основни класове. Производният клас може да наследи компонентите на един или няколко базови класа. В първия случай наследяването се нарича **единично (просто)**, а във втория – **множествено**.

Дефинирането на производни класове е еквивалентно на конструирането на йерархии от класове. Ако множество от класове имат общи данни и методи, тези общи части могат да се обособят като основни класове, а всяка от останалите части да се дефинира като производен клас на съответния основен клас. Така се прави икономия на памет, тъй като се избягва многократното описание на едни и същи програмни фрагменти.

При конструирането на производни класове е достатъчно да се разполага само с обектните модули на основните класове, а не с техния програмен код. Това позволява да бъдат създавани библиотеки от класове, които да бъдат използвани при създаването на производни класове.

17.1 Дефиниране на производни класове

Подобно на обикновените, производните класове се дефинират като се *декларира класът* и се *дефинират неговите методи*.

Деклариране на производен клас

```
<декларация_на_производен_клас> ::=

class <име_на-производен_клас> :

        [<атрибут_за_област>]опц <име_на_базов_клас>

        {, [<атрибут_за_област>]опц <име_на_базов_клас>}опц

{<декларация_на_компоненти>

};

<име_на-производен_клас> ::= <идентификатор>

<атрибут_за_област> ::= public | private | protected

<име_на_базов_клас> ::= <идентификатор>
```

Пред всяко име на базов клас *може* да се постави запазената дума public, private или protected. Нарича се **атрибут за област**, тъй като определя областта на наследените членове. Употребата на атрибутите за област е различна от тази за обявяване на секции в тялото на класа. Ако атрибут за област е пропуснат, подразбира се private. Атрибутът protected е включен в новите версии на езика и не се използва много често.

Примери:

1. Декларацията: class der : base1, base2, base3

{...

};

определя производен клас der с три основни класа base1, base2 и base3. Тъй като атрибутът за област е пропуснат и за трите базови класа, подразбира се private, т.е. декларацията е еквивалентна на

class der : private base1, private base2, private base3

{...

};

2. Декларацията: class der : public base1, base2, base3

{...

};

е еквивалентна на

class der : public base1, private base2, private base3

{...

};

3. Декларацията: class der : protected base1, base2, public base3

{...

};

е еквивалентна на

class der : protected base1, private base2, public base3

{...

};

Декларациите на компонентите на производен клас, а също дефинициите на неговите методи не се различават от съответните при обикновените класове.

Множеството от компонентите на един производен клас се състои от компонентите на неговите базови класове и компонентите, декларирани в самия производен клас. Оттук произлиза и терминът наследяване. Механизмът, чрез който производният клас получава компонентите на базовия, се нарича наследяване. Когато производният клас има няколко базови класа, той наследява компонентите на всеки от тях. Наследяването в този случай е множествено.

Процесът на наследяване се изразява в следното:

- наследяват се данните и методите на основния клас;
- получава се достъп до някои от наследените членове на основния клас;
- производният клас "познава" реализацията само на основния клас, от който произлиза;
- производният клас може да е основен за други класове.

Производният клас може да дефинира допълнително:

- свои член-данни;
- методи, аналогични на тези на основния клас, а също и нови.

Дефинираните в производния клас данни и методи се наричат **собствени**.

Производният клас е дефиниран след като вече е дефиниран базовият клас, от който той произлиза. Чрез него се разширява декларацията на съществуващ клас. Разширяемостта на класовете е една от важните характеристики на ООП.

17. 2 Наследяване и достъп до наследените компоненти

Ще напомним, че в рамките на един клас (без наследяване), protected частта има аналогична роля като тази на private частта. До компоненти от тип protected имат пряк достъп само член-функции и приятелски функции на класа.

Атрибутът за област на базовия клас в декларацията на производния клас (public, private или protected) управлява механизма на наследяване и определя какъв да бъде режимът на достъп до наследените членове. Таблицата от Фиг. 17.3 показва наследяванията на компоненти на основен клас в зависимост от атрибута за област.

Атрибут за област	**Компонента на основен клас, определена като**	**Наследява се като**
public	private public protected	Private public protected
Private	private public protected	Private private private
Protected	private public protected	Private protected protected

Фиг. 17.3 Наследявания на компоненти на основен клас в производен

От таблицата се вижда, че:

· Ако базовият клас е деклариран като public в производния клас, всички private, public и protected компоненти на базовия клас се наследяват съответно като private, public и protected компоненти на производния клас.

Пример: Ако

```
class base                                  class der1 : public base
{private: int b1;                           {private: int d1;
 protected: int b2;                          protected: int d2;
 public: int b3();                          public: int d3();
};                                                                  };
```

можем да си мислим, че der1 е клас от вида:

```
    class der1
{private:
 int b1;
 int d1;
protected:
 int b2;
 int d2;
public:
 int b3();
 int d3();
};
```

· Ако базовият клас е деклариран като private в производния клас, всички негови компоненти се наследяват като private.

Пример: Ако

```
class base                                  class der2 : private base
{private: int b1;                           {private: int d1;
 protected: int b2;                          protected: int d2;
 public: int b3();                          public: int d3();
};                                                                  };
```

можем да си мислим, че der2 е клас от вида:

```
    class der2
{private:
 int b1;
 int b2;
 int b3();
 int d1;
 protected:
 int d2;
 public:
 int d3();
};
```

· Ако базовият клас е деклариран като protected в производния клас, private компонентите му се наследяват като private, а public и protected – като protected.

Пример: Ако

```
class base                                  class der3 : protected base
```

```
{private: int b1;                    {private: int d1;
 protected: int b2;                   protected: int d2;
 public: int b3();                  public: int d3();
};                                                        };
```

можем да си мислим, че der3 е клас от вида:

```
   class der3
{private:
 int b1;
 int d1;
 protected:
 int b2;
 int d2;
 int b3();
 public:
 int d3();
};
```

Наследените компоненти обаче се различават от декларираните в производния клас по правата за достъп. Производният клас има пряк достъп до компонентите, декларирани като public и protected, но няма пряк достъп до декларираните като private в базовия клас. Достъпът до private компонентите на базовия клас се извършва чрез неговия интерфейс.

Таблицата от Фиг. 17.4 показва прекия достъп на член-функции на производния клас (ПД) и външния достъп на производния клас (ВД) до компонентите на базовия клас.

компонента на базов клас	производен клас с атрибут public		производен клас с атрибут private		производен клас с атрибут protected	
	ПД	ВД	ПД	ВД	ПД	ВД
Public	да	да	да	не	да	не
Protected	да	не	да	не	да	не
Private	не	не	не	не	не	не

Фиг. 17.4 Достъп на член-функции и на обекти на производния клас до компонентите на базовия клас

Ще се спрем на някои от често срещаните случаи за достъп до членове на производен и основен клас, а също на достъпа на външни функции до наследен компонент. Ще изкажем и някои правила за достъп до компоненти на базови и производни класове, които ще подкрепим с още примери.

· Достъп до членове на основен клас чрез дефиниции на методи на производен клас

В сила са следните правила за достъп:

- Методите на производен клас (без значение на атрибута за област) нямат директен достъп до членовете от private–секцията на основния му клас

- В дефинициите на собствени методи на производния клас могат да се използват методите от секциите public и protected на основния му клас

- В дефинициите на собствени методи на производния клас може директно да се използват член-данните на секцията protected на основния му клас

· Достъп до методи чрез обекти на основния и производния клас

Обект на основен клас има пряк достъп до всички свои компоненти, обявени като public и няма пряк достъп до компонентите, обявени като private и protected. Обект на производен клас има пряк достъп до public компонентите на собствения си и компонентите на основния клас, наследени в производния клас като public.

· Достъп на основен клас до членове на производен клас и обратно

Методите на основния клас нямат достъп до членове на производен клас. Причината е, че когато основният клас се дефинира, не е ясно какви производни класове ще произхождат от него.

Производният клас също няма привилигирован достъп до членове на основния клас. От фиг. 17.3 се вижда, че производните класове нямат достъп до методите, обявени като private в основния клас.

Допустими са редица присвоявания между обекти на основния и производния клас. Ще ги разгледаме подробно в следващи части на главата. Засега ще отбележим само, че за реализирането им се извършват редица преобразувания.

· Достъп на функции приятели на производен клас до компоненти на основния му клас

Ще напомним, че функциите-приятели на клас не са елементи на класа, на който са приятели. Те са външни функции, получили привилигирован достъп до компонентите на класа.

Функциите приятели на производен клас имат същите права на достъп като член-функциите на производния клас. *Имат пряк достъп до всички компоненти, декларирани в класа и до public и protected компонентите на основния клас. Декларацията за приятелство не се наследява. Функция приятел на базовия клас не е приятел (освен ако не е декларирана като такава) на производния клас*.